

**LAPORAN PRAKTIKUM
ALGORITMA & STRUKTUR DATA
MODUL 6**



Graph and Tree

Oleh:

Nuzulia Sekti Mahanania NIM. 2510817220006

**PROGRAM STUDI TEKNOLOGI INFORMASI
FAKULTAS TEKNIK
UNIVERSITAS LAMBUNG MANGKURAT
JUNI 2026**

LEMBAR PENGESAHAN
LAPORAN PRAKTIKUM ALGORITMA & STRUKTUR DATA
MODUL 6

Laporan Praktikum Algoritma & Struktur Data Modul 6: Graph and Tree ini disusun sebagai syarat lulus mata kuliah Praktikum Algoritma & Struktur Data. Laporan Praktikum ini dikerjakan oleh:

Nama Praktikan : Nuzulia Sekti Mahanania
NIM : 2510817220006

Menyetujui,
Asisten Praktikum

Mengetahui,
Dosen Penanggung Jawab Praktikum

Muhammad Fauzan Ahsani
NIM. 2310817310009

Ir. Muhamamd Alkaff, S.Kom, M.Kom, Ph.D
NIP. 198606132015041001

DAFTAR ISI

LEMBAR PENGESAHAN.....	i
DAFTAR ISI	ii
DAFTAR GAMBAR.....	iii
DAFTAR TABEL	iv
Soal 1: Konversi Adjacency List ke Adjacency Matrix	1
A. Source Code.....	1
B. Output Program	2
C. Pembahasan	3
Soal 2: Konversi Adjacency Matrix ke Adjacency List	4
A. Source Code.....	4
B. Output Program	5
C. Pembahasan	5
Soal 3: BFS: Jarak Terpendek Keluar Labirin.....	7
A. Source Code.....	7
B. Output Program	9
C. Pembahasan	9
Soal 4: DFS: Menghitung Semua Jalur Keluar Labirin.....	11
A. Source Code.....	11
B. Output Program	13
C. Pembahasan	13
Soal 5: Tree Traversal.....	16
A. Source Code.....	16
B. Output Program	18
C. Pembahasan	18
TAUTAN GITHUB.....	20

DAFTAR GAMBAR

Gambar 1. Screenshot Hasil Soal 1	2
Gambar 2. Screenshot Hasil Soal 2	5
Gambar 3. Screenshot Hasil Soal 3	9
Gambar 4. Screenshot Hasil Soal 4	13
Gambar 5. Screenshot Hasil Soal 5	18

DAFTAR TABEL

Tabel 1. Source Code Soal 1	1
Tabel 2. Source Code Soal 2	4
Tabel 3. Source Code Soal 3	7
Tabel 4. Source Code Soal 4	11
Tabel 5. Source Code Soal 5	16

Soal 1: Konversi Adjacency List ke Adjacency Matrix

A. Source Code

Tabel 1. Source Code Soal 1

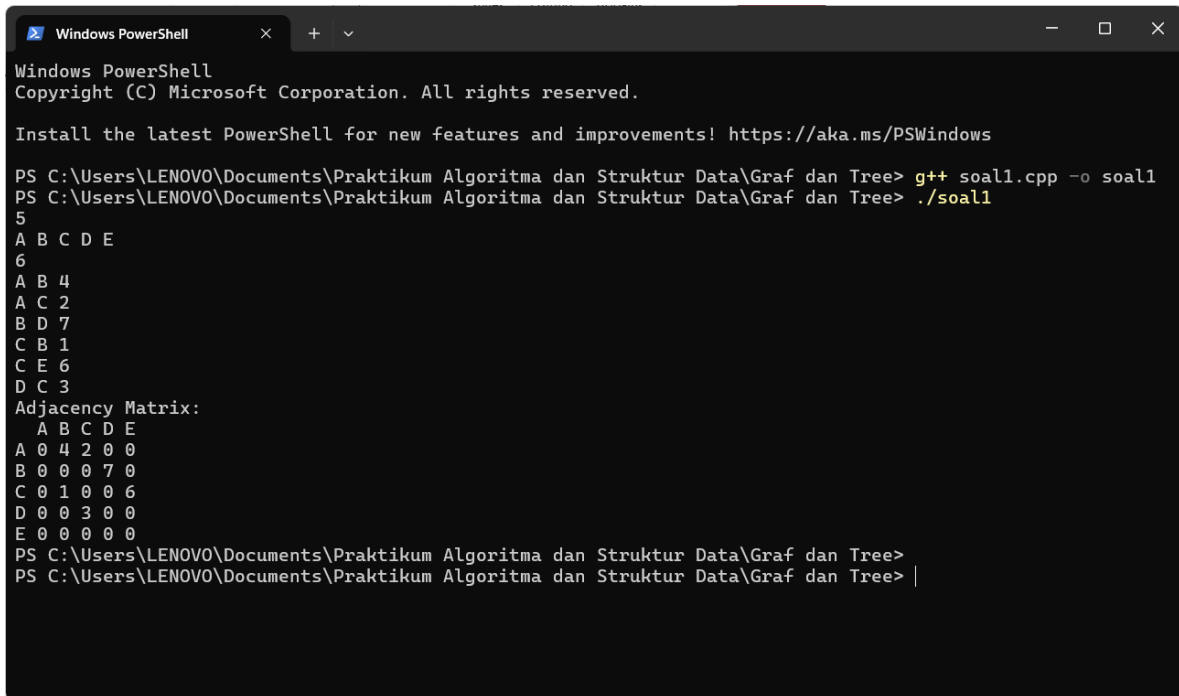
1	#include <iostream>
2	
3	using namespace std;
4	
5	int main() {
6	int N;
7	cin >> N;
8	
9	char vertex[10];
10	for (int i = 0; i < N; i++) {
11	cin >> vertex[i];
12	}
13	
14	int M;
15	cin >> M;
16	
17	int matrix[10][10];
18	for (int i = 0; i < N; i++) {
19	for (int j = 0; j < N; j++) {
20	matrix[i][j] = 0;
21	}
22	}
23	
24	for (int i = 0; i < M; i++) {
25	char U, V;
26	int W;
27	cin >> U >> V >> W;
28	
29	int indexU = 0, indexV = 0;
30	
31	for (int j = 0; j < N; j++) {
32	if (vertex[j] == U) indexU = j;
33	if (vertex[j] == V) indexV = j;
34	}
35	
36	matrix[indexU][indexV] = W;
37	}

```

38
39     cout << "Adjacency Matrix:\n ";
40     for (int i = 0; i < N; i++) {
41         cout << vertex[i];
42         if (i != N - 1) cout << " ";
43     }
44     cout << "\n";
45
46     for (int i = 0; i < N; i++) {
47         cout << vertex[i] << " ";
48         for (int j = 0; j < N; j++) {
49             cout << matrix[i][j];
50             if (j != N - 1) cout << " ";
51         }
52         cout << "\n";
53     }
54
55     return 0;
56 }

```

B. Output Program



```

Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\Users\LENOVO\Documents\Praktikum Algoritma dan Struktur Data\Graf dan Tree> g++ soal1.cpp -o soal1
PS C:\Users\LENOVO\Documents\Praktikum Algoritma dan Struktur Data\Graf dan Tree> ./soal1
5
A B C D E
6
A B 4
A C 2
B D 7
C B 1
C E 6
D C 3
Adjacency Matrix:
  A B C D E
A 0 4 2 0 0
B 0 0 0 7 0
C 0 1 0 0 6
D 0 0 3 0 0
E 0 0 0 0 0
PS C:\Users\LENOVO\Documents\Praktikum Algoritma dan Struktur Data\Graf dan Tree>
PS C:\Users\LENOVO\Documents\Praktikum Algoritma dan Struktur Data\Graf dan Tree> |

```

Gambar 1. Screenshot Hasil Soal 1

C. Pembahasan

Untuk penyelesaian Soal 1, kode ini dibikin buat ngubah representasi graf dari bentuk daftar *edge* ke dalam bentuk matriks dua dimensi berukuran $N \times N$. Pendekatannya lumayan simpel. Awalnya, program bakal inisialisasi matriks maksimal 10×10 dengan nilai 0 semua. Nilai 0 ini menandai kalau belum ada sisi atau jalur yang nyambung antar *vertex*. Terus, karena input *vertex*-nya berupa huruf alfabet, program butuh ngelakuin *sequential search* buat mencari indeks dari titik asal (U) dan titik tujuan (V) sebelum masukin bobotnya (W) ke koordinat sel matriks yang tepat. Asumsi utamanya mengikuti batasan dari soal, yaitu graf maksimal cuma punya 10 titik, sifatnya *directed* (searah), dan tiap label *vertex* dipastikan unik.

Kalau dianalisis dari segi kompleksitasnya, algoritma ini efisien banget buat batasan soal yang dikasih:

- Kompleksitas Waktu

Proses inisialisasi matriks awal dan cetak output butuh waktu $O(N^2)$. Pas *looping* untuk masukin sisi, ada pencarian indeks yang butuh $O(N)$ tiap kalinya buat sebanyak M sisi, jadinya memakan waktu $O(N \times M)$. Total kompleksitas waktunya kira-kira $O(N^2 + N \times M)$. Karena nilai N maksimal cuma 10, eksekusinya sudah pasti bakal cepat dan lolos dari *time limit* 1.0 detik.

- Kompleksitas Ruang

Memori yang dipakai mayoritas cuma dialokasiin buat deklarasi *array* nama *vertex* sama matriks. Jadi *space complexity*-nya cuma butuh $O(N^2)$. Alokasi segini tergolong sangat kecil buat melewati batasan *memory limit* 64 MB.

Berdasarkan hasil *testing* pake contoh kasus di soal, logikanya udah terbukti jalan dengan bener buat menangani *directed weighted graph*. Misalnya pas dikasih input sisi A B 4, program sukses ngubah sel di baris A dan kolom B jadi angka 4. Dan karena graf ini sifatnya searah, sel baliknya (baris B kolom A) tetep dipertahankan bernilai 0. Algoritma ini juga terbukti berhasil menangani titik yang ga punya rute keluar sama sekali (*vertex* E di contoh kasus), yang mana di hasil akhirnya seluruh baris E dibiarin tetep 0. Kesimpulannya, output matriksnya udah akurat dan sesuai persis sama permasalahan yang diminta.

Soal 2: Konversi Adjacency Matrix ke Adjacency List

A. Source Code

Tabel 2. Source Code Soal 2

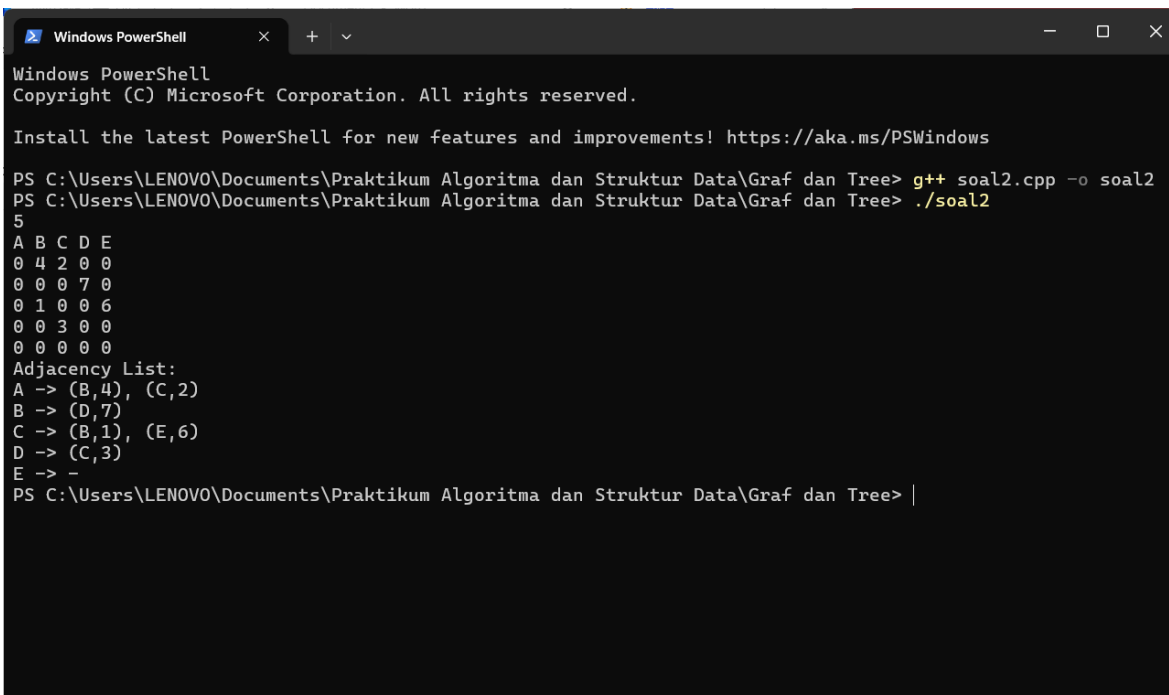
1	#include <iostream>
2	
3	using namespace std;
4	
5	int main() {
6	int N;
7	cin >> N;
8	
9	char vertex[10];
10	for (int i = 0; i < N; i++) {
11	cin >> vertex[i];
12	}
13	
14	int matrix[10][10];
15	for (int i = 0; i < N; i++) {
16	for (int j = 0; j < N; j++) {
17	cin >> matrix[i][j];
18	}
19	}
20	
21	cout << "Adjacency List:\n";
22	for (int i = 0; i < N; i++) {
23	cout << vertex[i] << " -> ";
24	
25	bool hasEdge = false;
26	
27	for (int j = 0; j < N; j++) {
28	if (matrix[i][j] > 0) {
29	if (hasEdge) {
30	cout << ", ";
31	}
32	cout << "(" << vertex[j] << ", " <<
33	matrix[i][j] << ")";
34	hasEdge = true;
35	}
36	}

```

37         if (!hasEdge) {
38             cout << "-";
39         }
40         cout << "\n";
41     }
42
43     return 0;
44 }

```

B. Output Program



```

Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\Users\LENOVO\Documents\Praktikum Algoritma dan Struktur Data\Graf dan Tree> g++ soal2.cpp -o soal2
PS C:\Users\LENOVO\Documents\Praktikum Algoritma dan Struktur Data\Graf dan Tree> ./soal2
5
A B C D E
0 4 2 0 0
0 0 0 7 0
0 1 0 0 6
0 0 3 0 0
0 0 0 0 0
Adjacency List:
A -> (B,4), (C,2)
B -> (D,7)
C -> (B,1), (E,6)
D -> (C,3)
E -> -
PS C:\Users\LENOVO\Documents\Praktikum Algoritma dan Struktur Data\Graf dan Tree>

```

Gambar 2. Screenshot Hasil Soal 2

C. Pembahasan

Untuk penyelesaian Soal 2, kode ini dibuat untuk menangani konversi dari representasi *adjacency matrix* ke dalam bentuk *adjacency list*. Pertama-tama, program bakal ngebaca input jumlah *vertex* (N) dan nyimpen label karakternya ke dalam *array*. Habis itu, program lanjut ngebaca nilai bobot ke dalam matriks dua dimensi. Nah, buat proses cetaknya, teknik utamanya pake *nested loop*. *Looping* baris yang di luar dipake buat nandain *vertex* asal, dan di tahap ini program bakal langsung nyetak karakter *vertex* tersebut ditambah tanda panah (->). Terus, *looping* yang di dalam bakal ngecek tiap kolom di baris itu. Kalau menemukan nilai sel yang lebih dari 0, itu nandain ada rute

berbobot, jadi program bakal nyetak *vertex* tujuan dan bobotnya dengan format (V, W) . Di dalam proses ini juga diselipin variabel boolean *flag* (yaitu *hasEdge*) buat menangani penambahan tanda koma secara dinamis biar outputnya rapi kalau ada lebih dari satu jalur. Asumsi yang dipake juga mengacu sama batesan soal, di mana program harus bisa menangani kondisi pas suatu titik ga punya *edge* keluar sama sekali. Kalau variabel boolean tadi tetep bernilai *false* sampai *looping* dalam kelar, program bakal otomatis nambahin tanda minus (-).

Kalau dilihat dari segi performanya, struktur algoritma ini udah sangat pas dan efisien buat ngerjain soalnya:

- Kompleksitas Waktu

Karena algoritma ini wajib memeriksa seluruh elemen sel di dalam matriks dua dimensi berukuran $N \times N$ tanpa terkecuali, maka kompleksitas waktu yang dibutuhkan adalah $O(N^2)$. Untuk batasan nilai N maksimal yang cuma 10, proses penelusuran dan pencetakan ini selesai cepat dan aman dari batasan *time limit* 1.0 detik.

- Kompleksitas Ruang

Memori yang dialokasikan pada program ini utamanya cuma dipakai buat nyimpen *array* label *vertex* berukuran 10 dan matriks berukuran 10×10 . Jadi, kompleksitas ruangnya adalah $O(N^2)$. Penggunaan memori ini tergolong kecil, jadinya ga ada masalah sama batasan *memory limit* 64 MB di soal.

Berdasarkan hasil testing pake contoh kasus uji, logika program ini udah terbukti bener dalam merepresentasikan *adjacency list* dari sebuah matriks. Sebagai pembuktian, misal pada baris A di matriks terdapat nilai 4 di kolom B dan nilai 2 di kolom C, program dengan sukses mencetak output $A \rightarrow (B, 4), (C, 2)$. Program ini juga terbukti berhasil menangani kondisi pas ketemu baris yang isinya angka 0 semua, contohnya kayak pada *vertex* E yang memang ga punya *edge* keluar sama sekali. Di hasil akhir, baris E sukses dicetak sebagai $E \rightarrow -$ tanpa merusak format baris lainnya. Ini menunjukkan kalau pemetaan graf berarah dan berbobot dari matriks ke bentuk list udah akurat dan sesuai sama spesifikasi permasalahan yang diminta.

Soal 3: BFS: Jarak Terpendek Keluar Labirin

A. Source Code

Tabel 3. Source Code Soal 3

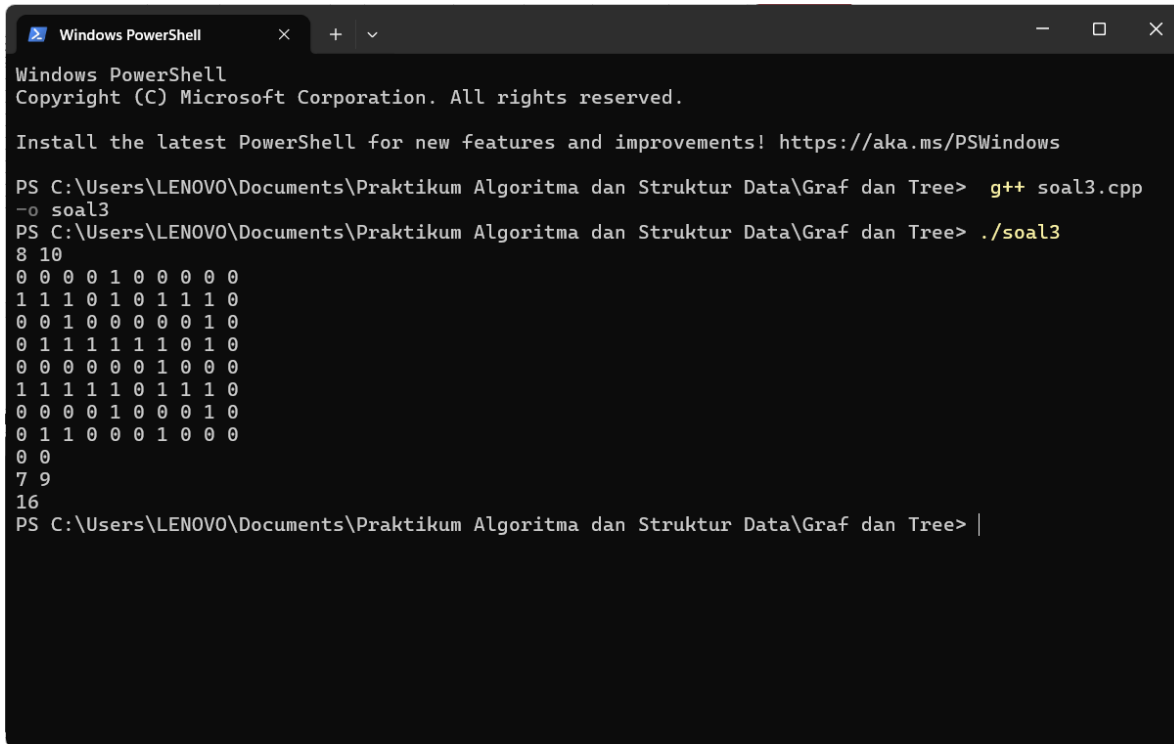
1	#include <iostream>
2	#include <queue>
3	
4	using namespace std;
5	
6	struct Position {
7	int row, column, distance;
8	};
9	
10	int G[105][105];
11	bool visited[105][105];
12	
13	int main() {
14	int R, C;
15	cin >> R >> C;
16	
17	for (int i = 0; i < R; i++) {
18	for (int j = 0; j < C; j++) {
19	cin >> G[i][j];
20	visited[i][j] = false;
21	}
22	}
23	
24	int SR, SC;
25	cin >> SR >> SC;
26	
27	int FR, FC;
28	cin >> FR >> FC;
29	
30	queue<Position> queue_positions;
31	queue_positions.push({SR, SC, 0});
32	visited[SR][SC] = true;
33	
34	int X = -1;
35	int direction_row[] = {-1, 1, 0, 0};
36	int direction_column[] = {0, 0, -1, 1};

```

37
38     while (!queue_positions.empty()) {
39         Position current = queue_positions.front();
40         queue_positions.pop();
41
42         if (current.row == FR && current.column == FC)
43         {
44             X = current.distance;
45             break;
46         }
47
48         for (int i = 0; i < 4; i++) {
49             int next_row = current.row +
direction_row[i];
50             int next_column = current.column +
direction_column[i];
51
52             if (next_row >= 0 && next_row < R &&
next_column >= 0 && next_column < C) {
53                 if (G[next_row][next_column] == 0 &&
!visited[next_row][next_column]) {
54                     visited[next_row][next_column] =
true;
55                     queue_positions.push({next_row,
next_column, current.distance + 1});
56                 }
57             }
58         }
59
60         cout << X << "\n";
61         return 0;
62     }

```

B. Output Program



```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\Users\LENOVO\Documents\Praktikum Algoritma dan Struktur Data\Graf dan Tree> g++ soal3.cpp
-o soal3
PS C:\Users\LENOVO\Documents\Praktikum Algoritma dan Struktur Data\Graf dan Tree> ./soal3
8 10
0 0 0 0 1 0 0 0 0 0
1 1 1 0 1 0 1 1 1 0
0 0 1 0 0 0 0 0 1 0
0 1 1 1 1 1 1 0 1 0
0 0 0 0 0 0 1 0 0 0
1 1 1 1 1 0 1 1 1 0
0 0 0 0 1 0 0 0 1 0
0 1 1 0 0 0 1 0 0 0
0 0
7 9
16
PS C:\Users\LENOVO\Documents\Praktikum Algoritma dan Struktur Data\Graf dan Tree> |
```

Gambar 3. Screenshot Hasil Soal 3

C. Pembahasan

Untuk penyelesaian Soal 3, kode ini dirancang buat nyari langkah minimum atau rute terpendek buat keluar dari sebuah labirin, yang mana paling cocok diselesaikan pake algoritma *Breadth-First Search* (BFS). Proses kerjanya bertahap. Pertama, program menyiapkan *array* dua dimensi berukuran maksimal 105x105 untuk nyimpan grid peta (G) dan *array* boolean *visited* buat nandain sel mana aja yang sudah pernah dilewatin. Karena BFS butuh struktur data antrean, program pakai *queue* buat nyimpen data koordinat baris, kolom, dan jumlah jarak (langkah) dari titik awal. Awalnya, titik *start* (SR, SC) akan dimasukin ke dalem antrean dengan jarak 0 dan langsung ditandain *true* di *array* *visited*. Lalu, program bakal masuk ke *looping* utama buat ngeluarin titik paling depan dari antrean dan ngecek 4 arah pergerakan (atas, bawah, kiri, kanan). Kalau titik pergerakan selanjutnya ternyata valid (masih di dalem area grid, nilainya 0 yang berarti bisa dilewati, dan belom pernah dikunjungi), titik itu bakal dimasukin ke dalem antrean dan jaraknya di-plus 1. Proses ini bakal terus berulang sampai antrean kosong atau koordinat saat ini udah sama persis sama titik tujuan (FR, FC). Asumsi dari algoritma

ini adalah labirin ga punya arah diagonal, pergerakan terbatas pada 4 arah, dan variabel default jarak X udah di-set -1 dari awal buat jaga-jaga kalau jalan keluarnya beneran buntu.

Kalau dianalisis dari segi efisiensinya, algoritma BFS ini udah merupakan solusi paling optimal buat kasus pencarian rute terpendek di *unweighted graph*:

- Kompleksitas Waktu

Pada kasus terburuk (*worst case*), algoritma BFS ini bakal ngunjungi setiap sel di dalam peta labirin tepat satu kali. Oleh karena itu, kompleksitas waktu yang dibutuhkan berbanding lurus sama luas petanya, yaitu sebesar $O(R \times C)$, dengan R adalah jumlah baris dan C adalah jumlah kolom. Karena ukuran maksimal R dan C di soal adalah 100, algoritma ini berjalan efisien dan ga kena masalah *time limit*.

- Kompleksitas Ruang

Memori yang dipake mayoritas teralokasi untuk menyimpan grid matriks 105×105 , *array visited* 105×105 , serta memori dinamis untuk struktur antrean (*queue*) yang ukuran maksimalnya sebesar jumlah sel yang bisa dilewati. Jadi *space complexity*-nya adalah $O(R \times C)$. Ukuran ini juga masih sangat proporsional dan bakal lolos ngejalanin *memory limit*.

Berdasarkan analisis kasus yang dikasih, logika BFS ini terbukti sangat akurat dalam mencari rute. Sifat dasar dari BFS ngejamin kalau titik tujuan (FR, FC) pertama kali ditemui di dalam *queue*, rute tersebut pastilah rute yang paling optimal karena pencariannya tersebar secara merata per lapis. Pas program nyentuh titik akhir, dia bakal langsung *break* dan nyimpen langkah minimalnya ke variabel X. Algoritma ini juga terbukti berhasil menangani kondisi ekstrim di mana jalan keluarnya bener-bener ketutup rintangan semua atau ga ada jalur sama sekali yang menghubungkan *start* dan *finish*. Dalam kasus kayak gini, karena *looping* selesai tanpa pernah nyentuh titik akhir, program bakal tetep mencetak nilai bawaan dari variabel X yaitu -1, yang secara akurat merepresentasikan kalau solusi jalurnya emang ga ada. Jadi secara pembuktian metrik, jalannya program udah sangat pas dengan ekspektasi masalahnya.

Soal 4: DFS: Menghitung Semua Jalur Keluar Labirin

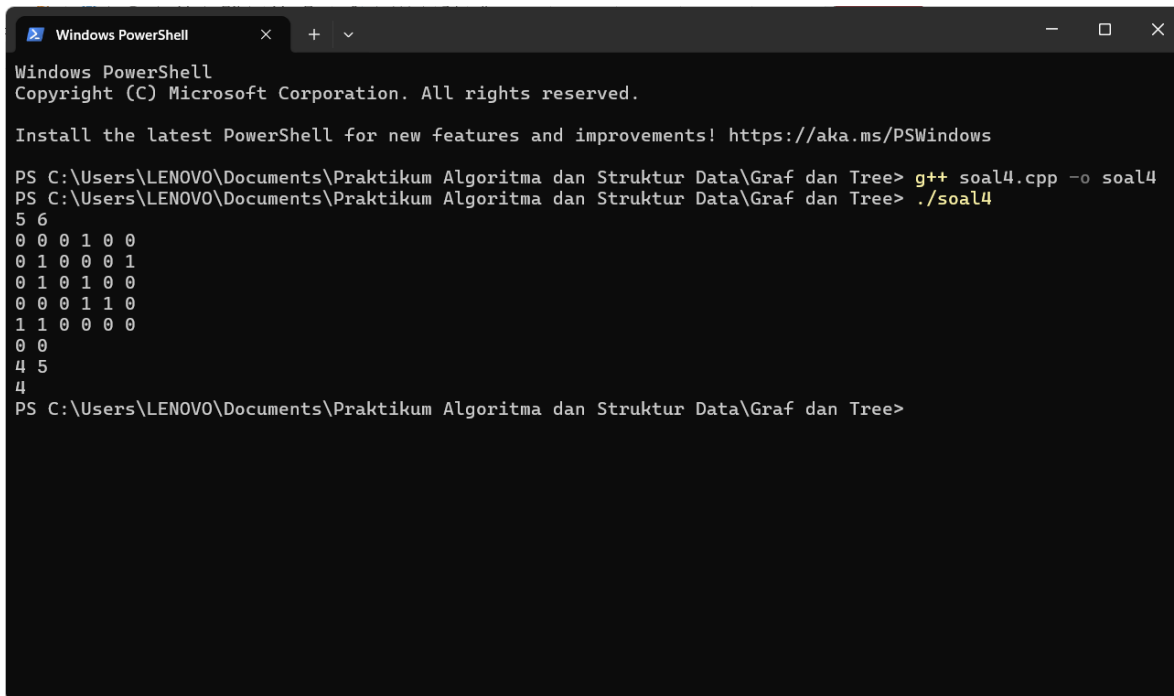
A. Source Code

Tabel 4. Source Code Soal 4

1	#include <iostream>
2	
3	using namespace std;
4	
5	int R, C;
6	
7	int G[10][10];
8	bool visited[10][10];
9	
10	int FR, FC;
11	int T = 0;
12	
13	int direction_row[] = {-1, 1, 0, 0};
14	int direction_column[] = {0, 0, -1, 1};
15	
16	void dfs(int row, int column) {
17	if (row == FR && column == FC) {
18	T++;
19	return;
20	}
21	
22	visited[row][column] = true;
23	
24	for (int i = 0; i < 4; i++) {
25	int next_row = row + direction_row[i];
26	int next_column = column + direction_column[i];
27	
28	if (next_row >= 0 && next_row < R && next_column
29	>= 0 && next_column < C) {
30	if (G[next_row][next_column] == 0 &&
31	!visited[next_row][next_column]) {
32	dfs(next_row, next_column);
33	}
34	}


```
35     visited[row][column] = false;
36 }
37
38 int main() {
39     cin >> R >> C;
40
41     for (int i = 0; i < R; i++) {
42         for (int j = 0; j < C; j++) {
43             cin >> G[i][j];
44             visited[i][j] = false;
45         }
46     }
47
48     int SR, SC;
49     cin >> SR >> SC;
50     cin >> FR >> FC;
51
52     dfs(SR, SC);
53
54     cout << T << "\n";
55     return 0;
56 }
```

B. Output Program



```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\Users\LENOVO\Documents\Praktikum Algoritma dan Struktur Data\Graf dan Tree> g++ soal4.cpp -o soal4
PS C:\Users\LENOVO\Documents\Praktikum Algoritma dan Struktur Data\Graf dan Tree> ./soal4
5 6
0 0 0 1 0 0
0 1 0 0 0 1
0 1 0 1 0 0
0 0 0 1 1 0
1 1 0 0 0 0
0 0
4 5
4
PS C:\Users\LENOVO\Documents\Praktikum Algoritma dan Struktur Data\Graf dan Tree>
```

Gambar 4. Screenshot Hasil Soal 4

C. Pembahasan

Untuk penyelesaian Soal 4, kode ini dirancang buat menghitung total seluruh jalur yang mungkin dilewat dari titik awal ke titik tujuan di dalam labirin. Algoritma yang paling tepat untuk menangani permasalahan ini adalah *Depth-First Search* (DFS) yang dikombinasikan dengan teknik *backtracking*. Proses kerjanya dimulai dengan membaca batesan ukuran grid R dan C, representasi matriks labirin (G), serta koordinat awal (SR, SC) dan akhir (FR, FC). Di dalam fungsi `dfs(row, column)`, program bakal melakukan penelusuran rute sedalam mungkin ke satu cabang jalur terlebih dahulu. Tiap kali program menginjak suatu sel, sel tersebut langsung ditandai `true` di array boolean `visited` supaya penelusuran di cabang rute yang sama ga *looping* di titik yang sama. Dari titik itu, program bakal ngecek validitas pergerakan ke 4 arah pakai bantuan array `direction_row` dan `direction_column`. Kalau sel nextnya bernilai 0 (bisa dilewat) dan statusnya belum pernah dikunjungi, fungsi `dfs` bakal dipanggil lagi secara rekursif buat ngelanjutin penelusuran rute tersebut. Kunci utamanya ada di bagian akhir fungsi rekursif, yaitu proses *backtracking*. Pas suatu cabang penelusuran udah mentok (nemu jalan buntu) atau selesai dieksplorasi, program bakal mengembalikan

status `visited` sel tersebut menjadi `false` lagi sebelum fungsi keluar. Tujuannya adalah supaya sel itu bisa digunain lagi oleh cabang rute alternatif lain yang mau lewat situ. Setiap kali langkah rekursif berhasil menyentuh target koordinat akhir (FR, FC), nilai pada variabel global T bakal otomatis bertambah satu (+1) sebagai tanda satu jalur sederhana telah ditemukan. Asumsi dari algoritma ini adalah penelusuran hanya mencari jalur sederhana dan rute diagonal tidak dihitung.

Jika dianalisis dari segi performanya, sruktur algoritma ini udah pas buat menangani batasan yang diberikan di soal.

- Kompleksitas Waktu

Karena algoritma ini menggunakan teknik *backtracking* untuk mengeksplorasi seluruh kemungkinan kombinasi rute yang ada di dalam grid, secara teori kompleksitas waktunya bersifat eksponensial pada kasus terburuk (*worst case*), yaitu berkisar $O(4^{(R \times C)})$. Namun, karena batasan nilai R dan C di soal sangat kecil (maksimal cuma 7x7 atau 49 sel), total variasi jalurnya ga bakal terlalu banyak. Proses pencarian ini dipastikan bakal selesai cepat dan pastinya aman dari batasan *time limit* 1.0 detik.

- Kompleksitas Ruang

Alokasi memori pada program ini sangat hemat karena cuma dipake buat nyimpen matriks G ukuran 10x10 dan array `visited` ukuran 10x10. Selain itu, penggunaan memori tambahan cuma berasal dari *recursion stack* saat fungsi DFS dipanggil, yang mana kedalaman maksimalnya sebanding dengan jumlah total sel dalam labirin. Jadi, kompleksitas ruangnya adalah $O(R \times C)$. Penggunaan memori ini kecil dan aman dari batasan *memory limit* 64 MB.

Berdasarkan hasil testing menggunakan kasus uji, logika DFS dan *backtracking* ini terbukti bener dan akurat dalam menghitung variasi jalur. Sebagai pembuktian, mekanisme *backtracking* terbukti sukses membersihkan kembali jejak langkah (`visited = false`) tiap kali rute pencarian mundur dari jalan buntu. Hal ini membuat cabang penelusuran yang lain tetep bisa nemu jalur baru yang berbeda walaupun sempat melewati sel yang sama. Begitu fungsi rekursif menyentuh koordinat tujuan, variabel T sukses mencatat pertambahan jalur secara tepat. Program ini juga terbukti berhasil

menangani kondisi khusus pas posisi *start* dan *finish* benar-bener terisolasi oleh tembok (nilai 1). Dalam kasus buntu total seperti itu, fungsi dfs bakal kelar tanpa pernah memicu kondisi `row == FR && column == FC`, sehingga hasil akhir yang dicetak di layar tetap bernilai 0 sesuai sama ekspetasi permasalahan yang diminta di soal.

Soal 5: Tree Traversal

A. Source Code

Tabel 5. Source Code Soal 5

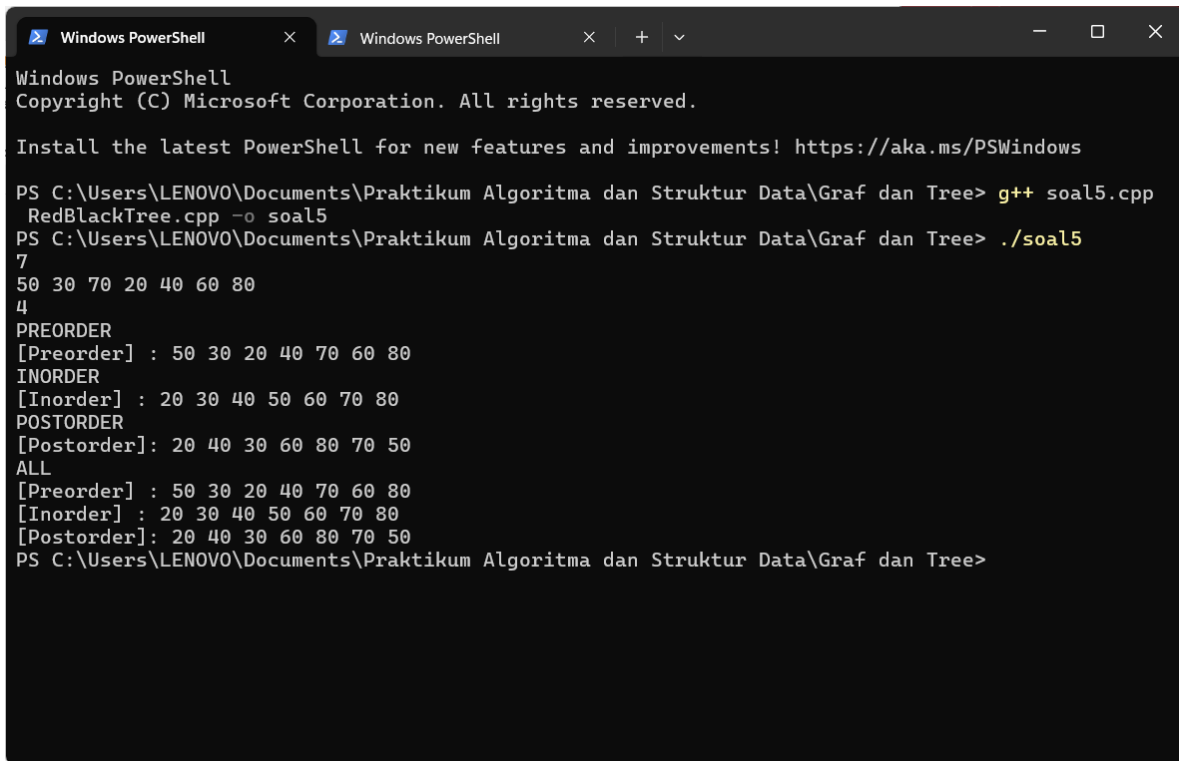
1	#include <iostream>
2	#include <string>
3	#include "RedBlackTree.h"
4	
5	using namespace std;
6	
7	void print_preorder(const RedBlackTree::Node* node) {
8	if (node->isNil) return;
9	
10	cout << " " << node->key;
11	print_preorder(node->left);
12	print_preorder(node->right);
13	}
14	
15	void print_inorder(const RedBlackTree::Node* node) {
16	if (node->isNil) return;
17	
18	print_inorder(node->left);
19	cout << " " << node->key;
20	print_inorder(node->right);
21	}
22	
23	void print_postorder(const RedBlackTree::Node* node) {
24	if (node->isNil) return;
25	
26	print_postorder(node->left);
27	print_postorder(node->right);
28	cout << " " << node->key;
29	}
30	
31	int main() {
32	int N;
33	cin >> N;
34	
35	RedBlackTree rbt;
36	for (int i = 0; i < N; i++) {

```

37         int A;
38         cin >> A;
39
40         if (!rbt.contains(A)) {
41             rbt.insert(A);
42         }
43     }
44
45     int Q;
46     cin >> Q;
47
48     for (int i = 0; i < Q; i++) {
49         string T;
50         cin >> T;
51
52         if (rbt.empty()) {
53             cout << "Tree kosong. Tidak ada yang bisa
ditampilkan.\n";
54             continue;
55         }
56
57         if (T == "PREORDER" || T == "ALL") {
58             cout << "[Preorder] :";
59             print_preorder(rbt.root());
60             cout << "\n";
61         }
62         if (T == "INORDER" || T == "ALL") {
63             cout << "[Inorder] :";
64             print_inorder(rbt.root());
65             cout << "\n";
66         }
67         if (T == "POSTORDER" || T == "ALL") {
68             cout << "[Postorder]:";
69             print_postorder(rbt.root());
70             cout << "\n";
71         }
72     }
73
74     return 0;
75 }

```

B. Output Program



```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\Users\LENOVO\Documents\Praktikum Algoritma dan Struktur Data\Graf dan Tree> g++ soal5.cpp
RedBlackTree.cpp -o soal5
PS C:\Users\LENOVO\Documents\Praktikum Algoritma dan Struktur Data\Graf dan Tree> ./soal5
7
50 30 70 20 40 60 80
4
PREORDER
[Preorder] : 50 30 20 40 70 60 80
INORDER
[Inorder] : 20 30 40 50 60 70 80
POSTORDER
[Postorder]: 20 40 30 60 80 70 50
ALL
[Preorder] : 50 30 20 40 70 60 80
[Inorder] : 20 30 40 50 60 70 80
[Postorder]: 20 40 30 60 80 70 50
PS C:\Users\LENOVO\Documents\Praktikum Algoritma dan Struktur Data\Graf dan Tree>
```

Gambar 5. Screenshot Hasil Soal 5

C. Pembahasan

Untuk penyelesaian Soal 5, kode ini dirancang buat menangani operasi pembuatan struktur data *Red-Black Tree* (RBT) sekaligus melakukan penelusuran *traversal* di dalam pohonnya. Secara definisi, RBT itu semacam *Binary Search Tree* yang punya kemampuan menyeimbangkan dirinya sendiri biar kedalamannya tetap optimal. Proses kerjanya dibagi jadi dua fase utama. Pertama, program bakal ngebaca N bilangan dan masukan bilangannya satu per satu ke dalam pohon. Pas proses ini, diselipin juga teknik pengecekan pakai fungsi `contains` buat menangani input nilai duplikat, jadi angka yang udah ada di dalam pohon ga bakal dimasukin dua kali. Fase kedua adalah ngebaca perintah *query*. Nah, di tahap ini program dibikin interaktif, yang artinya tiap kali program kelar ngebaca satu perintah *query* (T) dari user, saat itu juga program langsung mengeksekusi cetakan `PREORDER`, `INORDER`, `POSTORDER`, atau `ALL` untuk pemanggilan fungsi rekursif. Jadi program ga perlu repot simpan perintahnya dulu. Asumsi yang dibangun ngikutin aturan soal, di mana maksimal *query* (Q) adalah 20, dan

penanda *node* kosong diasumsikan ga sekedar pake *null* biasa, tapi pake properti `isNil` bawaan dari strukturnya.

Kalau dianalisis dari segi efisiensinya, algoritma struktur data RBT ini jauh lebih stabil buat menangani data dibanding *Binary Tree* biasa.

- Kompleksitas Waktu

Untuk setiap proses masukan data (*insert*) dan ngecek duplikat (*contains*) ke dalam RBT, waktu yang dibutuhkan adalah $O(\log N)$. Karena ada N data yang diinput, maka fase pertama butuh waktu $O(N \log N)$. Pada fase kedua, setiap eksekusi traversal bakal ngelewatin semua *node* yang ada, jadinya butuh waktu $O(N)$. Karena terdapat Q buah *query*, waktu untuk cetaknya adalah $O(Q \times N)$. Total keseluruhan kompleksitas waktunya $O(N \log N + Q \times N)$. Ini udah optimal dan bakal selesai di bawah *time limit* 1.0 detik.

- Kompleksitas Ruang

Memori utama cuma dialokasiin buat ngebangun struktur *node* di dalem pohon sebanyak N elemen. Karena di kodingan ini diterapin eksekusi *query* secara langsung dan ga butuh *array buffer* tambahan buat nyimpen string, maka kompleksitas ruangnya $O(N)$. Penggunaan memori ini sangat minim dan ga ngelewatin batasan *memory limit* 64 MB dari sistem.

Berdasarkan hasil eksekusi pake contoh soal, logika RBT dan mekanisme *input-output* interaktif ini terbukti akurat. Sebagai pembuktian, eksekusi *query* yang langsung dicetak sesaat setelah diinput terbukti jalan dengan mulus tanpa merusak alur pembacaan data. Terus, karena pada dasarnya RBT tetep mempertahankan sifat *Binary Search Tree*, pas program diminta ngejalanin perintah `INORDER`, output angkanya kebukti selalu tercetak urut dari yang terkecil ke yang terbesar. Logika pencegahan duplikat juga terbukti berhasil menangani angka kembar, misal kalau kita masukan angka 50 dua kali, cuma satu angka 50 yang kesimpen dan dicetak. Terakhir, program juga kebukti sukses menangani kondisi khusus, misalnya kalau kita ngasih *query* tapi pohonnya ga punya isian apa-apa, program ga bakal *crash* melainkan berhasil nyetak pesan "Tree kosong" sesuai sama yang diinstruksikan. Kesimpulannya, alur kerjanya udah sesuai persis sama ekspektasi permasalahan di soal.

TAUTAN GITHUB

<https://github.com/DSA26-ULM/module-6-graph-and-tree-nullrym>